

SLAM

INF8422: Robotic Perception and Spatial Intelligence

Prof. Pierre-Yves Lajoie

**POLYTECHNIQUE
MONTREAL**



Agenda

1. Introduction to SLAM
2. Cameras
3. Visual Odometry
4. IMU and Inertial Navigation
5. Visual-Inertial SLAM (VIO)
6. History of SLAM

Introduction to SLAM

What is SLAM ?

Simultaneous Localization And Mapping

- **Simultaneous** : Both processes run at the same time.
- **Localization** : Estimate the robot's pose (position and orientation) within the environment.
- **Mapping** : Build a model (a map) of the environment.

Fundamental objective

Make a robot **autonomous** in an **unknown** environment, without needing any external infrastructure (GPS, beacons, etc.).

The map and the trajectory are **estimated jointly** from the sensor data :

$$P(X, M \mid Z)$$

- $X = \{x_0, x_1, \dots, x_k\}$: Trajectory (poses).
- $M = \{l_1, \dots, l_m\}$: Map (landmarks).
- Z : Sensor observations.

The problem

If I know my **exact position** → building the map is easy (*mapping with known poses*).

If I know the **exact map** → localizing myself is easy (*localization on a known map*).

The SLAM dilemma

We know **neither of the two** at the start.

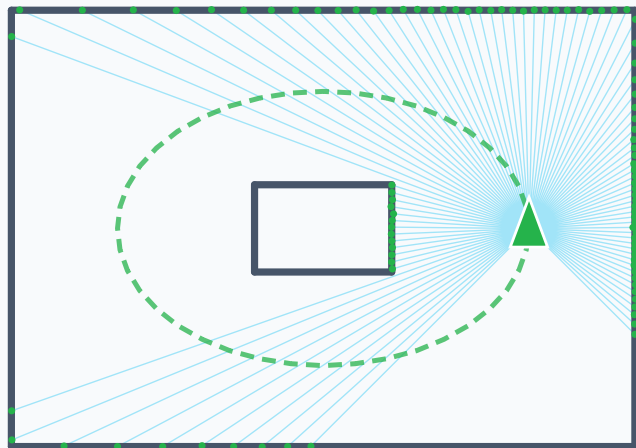
Localization errors *corrupt* the map, and map errors *degrade* localization.

Why is it hard?

1. **Sensor noise:** No measurement is perfect.
2. **Drift:** Uncertainty accumulates over time.
3. **Data association:** Is what I see a known place or a new one?
4. **Dynamic environments:** The world moves.
5. **Complexity:** The problem grows over time.

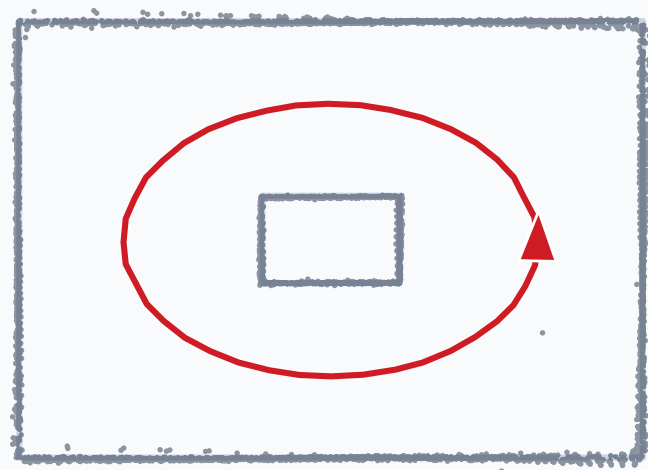
2D LiDAR SLAM

vérité terrain — le robot et son scan courant



--- trajectoire vraie faisceau lidar murs réels

carte estimée — scans posés aux poses estimées



— trajectoire estimée carte accumulée murs réels

$\sigma_{\text{scan}} = 0.01 \text{ m}$  FOV = 270° 96 rayons  $\sigma_{\text{odom}} = 0.03 \text{ m/pas}$     

k = 40 / 40 erreur de pose 0.05 m · épaisseur des murs 0.02 m

SLAM Inputs and Outputs

Inputs – Sensor Data

Proprioception (*internal measurements*) :

- Wheel encoders
- Inertial Measurement Unit (IMU)

Exteroception (*measurements of the environment*) :

- Cameras (mono, stereo, RGB-D)
- LiDAR (3D, 2D)
- Radar, Sonar

Outputs – Estimates

Robot trajectory:

$$X = \{x_0, x_1, \dots, x_k\} \in SE(3)^k$$

Map of the environment:

$$M = \{l_1, l_2, \dots, l_m\}$$

Points (sparse), voxels (dense), objects (semantic)...

Blind navigation vs SLAM

Dead Reckoning (no map)

- Integration of motion measurements (IMU, wheels).
- Errors accumulate over time.

Without any external correction, the robot eventually no longer knows where it is.

SLAM (with loop closures)

- Uses landmarks from the environment to correct the estimate.
- Every time a **known place is recognized**, the accumulated error is **bounded**.
- The trajectory and the map are **corrected globally**.

Example: the robot vacuum

As it moves through a room, it recognizes that it is passing near the charger again and re-registers its position.

Loop Closure

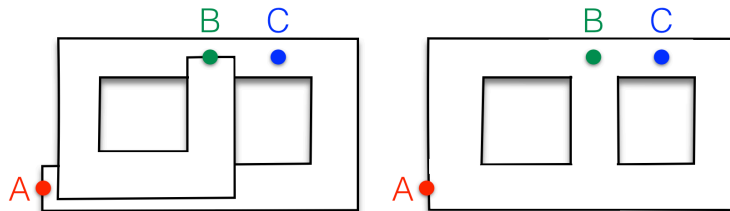
A **loop closure** occurs when the robot recognizes a place it has **already visited**.

This is the crucial moment when we can:

- **Bound** the accumulated error.
- **Correct** the entire past trajectory.
- **Merge** two parts of the map.

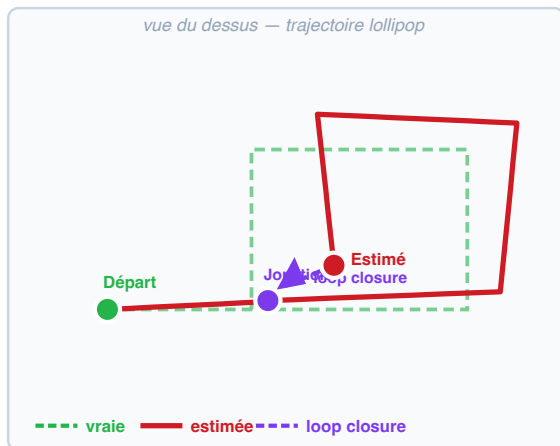
Without loop closure

The world looks like an "infinite corridor": the robot never realizes that it has returned to a known point.



Without vs with loop closure

Loop Closure – Interactive Visualization



$\sigma_{\text{drift}} = 1.5$ m



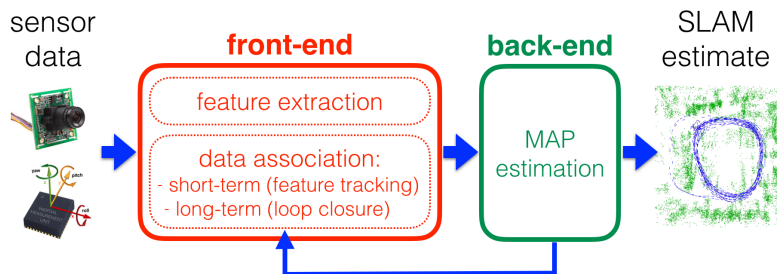
Détecter Loop Closure



Erreur de fermeture : 1.20 m → 1.20 m

Architecture: Front-end / Back-end

The standard breakdown of every modern SLAM system.



Front-end

Turns raw sensor data into **geometric constraints**:

- Feature extraction.
- Data association (tracking, loop detection).
- Local motion estimation.

Back-end

Takes the constraints from the Front-end and solves the **global problem**:

- Maximum A Posteriori (MAP).
- Factor graph optimization.
- Global correction of the trajectory and map.

Sensor Modalities for SLAM

Sensor	Strengths	Limitations
Monocular camera	Lightweight, cheap, texture-rich	No absolute scale, sensitive to lighting
Stereo / RGB-D	Direct depth, metric scale	Limited range (RGB-D < 5m)
3D LiDAR	Very accurate, works in darkness	High cost, sparse
IMU	High-frequency 6-DOF (100-1000 Hz)	Drift, bias

Sensor Modalities for SLAM

Fusion	Main advantage
Visual SLAM	Textural richness, compact
LiDAR SLAM	Absolute geometric precision
VIO (Visual-Inertial)	Robustness to blur, fast motion
LIO (LiDAR-Inertial)	Precision + corrected distortion
Radar SLAM	Fog, rain, night

Robustness

The most robust systems combine **several modalities**: VIO + LiDAR for autonomous driving, camera + radar for extreme conditions.

Map Representations

Sparse

Keypoints / landmarks – sufficient for localization, lightweight.

Example: ORB-SLAM3

Dense

Full 3D reconstruction (mesh, voxels, TSDF) – required for navigation, manipulation, visualization.

Examples: KinectFusion, ElasticFusion

Semantic

Labeled objects (chair, door, wall) – scene understanding, stable landmarks for long-term localization.

Examples: SemanticFusion, Kimera

Topological

Place graph (node A connected to node B) – high-level navigation, compact.

The choice of representation depends on the **task**: localization → sparse; planning → dense; interaction → semantic.

This course: the Visual and Inertial Front-end

Section	Content	Role in the system
2 – Cameras	Pinhole model, calibration, stereo, RGB-D	Sensory input of the Front-end
3 – Visual Odometry	Features, geometry, local BA	Local motion estimation
4 – IMU	Accelerometer, gyroscope, integration	High-frequency proprioceptive sensor
5 – VIO	IMU factor, fusion, observability	Bridge Front-end → Back-end

Cameras

Pinhole Model: Perspective Projection

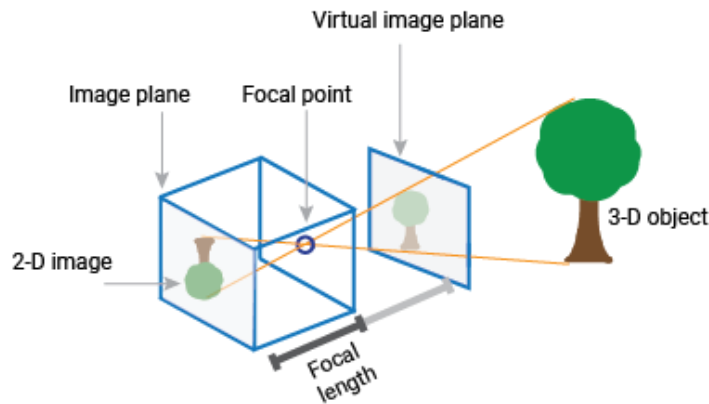
The pinhole model describes how a 3D point projects onto the image plane.

Principle: The image forms at a focal distance F (in mm) from the focal point.

Projection: Let $p^c = [p_x^c, p_y^c, p_z^c]^T$ be a point expressed in the camera frame (c). Its physical coordinates on the sensor are:

$$x = F \frac{p_x^c}{p_z^c}, \quad y = F \frac{p_y^c}{p_z^c}$$

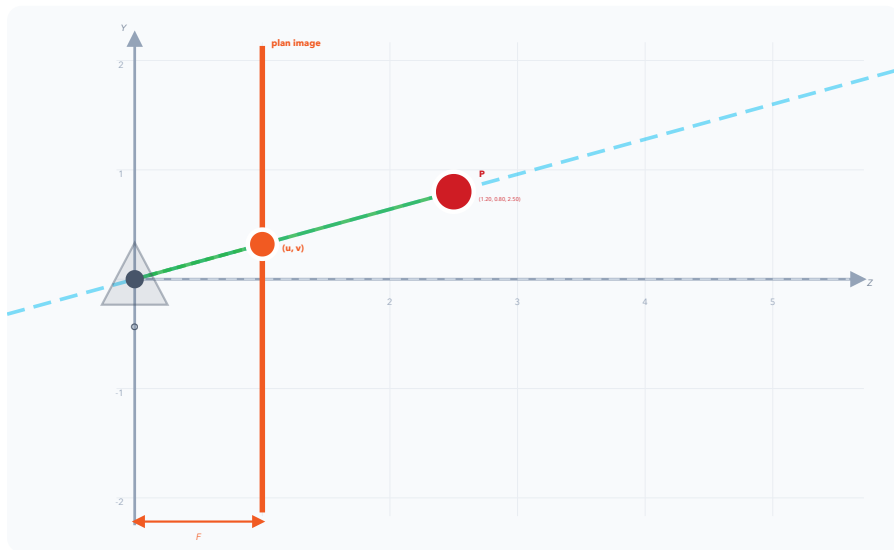
Note: x and y are physical distances (mm), not yet pixels.



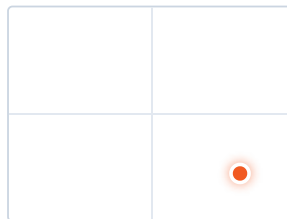
Geometric projection (Source: MathWorks)

This is the **perspective projection**: distant objects appear small. Depth information is **lost** in the projection.

Pinhole Model – Interactive Projection



Plan image (640 × 480 px)



(512, 368) px

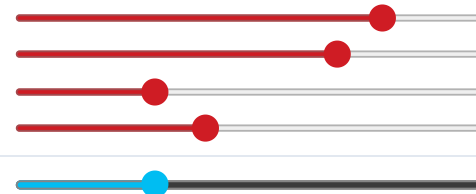
X = 1.20

Y = 0.80

Z = 2.50

f = 400 px

$\lambda = 1.00$



← déplacer P le long du rayon

$$u = f \cdot \frac{X}{Z} + c_x = 400 \cdot \frac{1.20}{2.50} + 320 = 512$$

$$v = f \cdot \frac{Y}{Z} + c_y = 400 \cdot \frac{0.80}{2.50} + 240 = 368$$

Δ λ s'annule : (u,v) ne change pas avec λ .
La profondeur Z est perdue.

Intrinsic Parameters: Camera $\rightarrow$ Pixels (u, v)

Converting physical coordinates (mm) to pixels (u, v) .

The focal length F (mm) is converted to pixels via the pixel size (p_u, p_v) :

$$f_u = \frac{F}{p_u}, \quad f_v = \frac{F}{p_v}$$

Intrinsic Matrix K :

$$\underbrace{\begin{bmatrix} f_u & 0 & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}}_K \begin{bmatrix} p_x^c/p_z^c \\ p_y^c/p_z^c \\ 1 \end{bmatrix}$$

- (f_u, f_v) : Focal lengths in pixels.
- (c_u, c_v) : Principal point, optical center of the image in pixels.

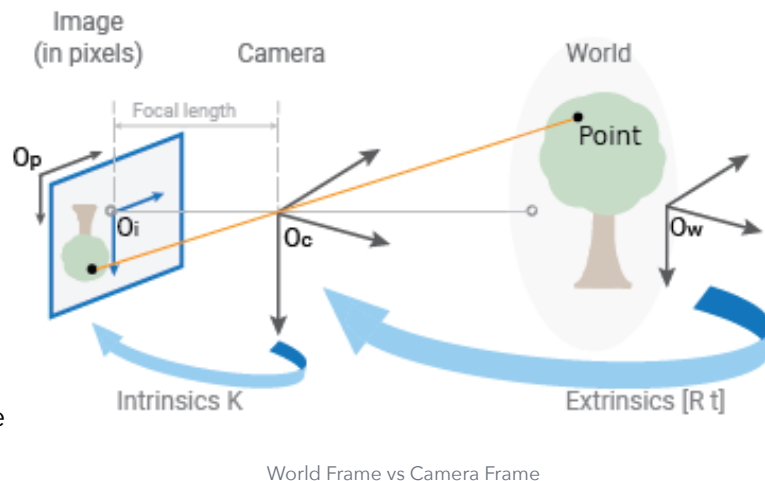
Extrinsic Parameters: World $\rightarrow$ Camera

Mapping from the world frame (w) to the camera frame (c).

Rigid transformation composed of a rotation R_w^c and a translation t_w^c :

$$\underbrace{\begin{bmatrix} R_w^c & t_w^c \\ 0 & 1 \end{bmatrix}}_{T_w^c} \begin{bmatrix} p^w \\ 1 \end{bmatrix}$$

- R_w^c (3×3), t_w^c (3×1) : rotation and translation **world $\rightarrow$ camera**.
- The **pose of the camera in the world** is the inverse (course 1) : $T_c^w = (T_w^c)^{-1}$, i.e. $R_c^w = (R_w^c)^T$ and $t_c^w = -(R_w^c)^T t_w^c$.
- In **SLAM** : it is this pose T_c^w (or its inverse T_w^c) that is the **unknown to be estimated**.



World Frame vs Camera Frame

Observation function $h(\cdot)$

In visual SLAM, the **observation function** relates a 3D point in the world to a pixel in the image:

Step 1 of $h(\cdot)$ – Extrinsics (World $\rightarrow$ Camera):

$$p^c = R_w^c p^w + t_w^c$$

Step 2 of $h(\cdot)$ – Intrinsics (Camera $\rightarrow$ Pixels):

$$\begin{bmatrix} u \\ v \end{bmatrix} = \underbrace{\begin{bmatrix} f_u & 0 & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}}_K \begin{bmatrix} p_x^c/p_z^c \\ p_y^c/p_z^c \\ 1 \end{bmatrix} = \begin{bmatrix} f_u \frac{p_x^c}{p_z^c} + c_u \\ f_v \frac{p_y^c}{p_z^c} + c_v \end{bmatrix}$$

We can therefore model an observation:

$$\bar{z} = h(T_c^w, p^w) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \Sigma)$$

- T_c^w : Camera pose (extrinsic parameter).
- p : 3D position of the landmark.
- $\bar{z} = (u, v)$: Observation in pixels.
- ϵ : noise.

Reprojection Error

The reprojection error for a point is $\|\bar{z} - h(T_c^w, p^w)\|^2$. The Back-End (next course) will then find the $T_{c_i}^w$ and p_i^w that minimize the sum of reprojection errors for all points:

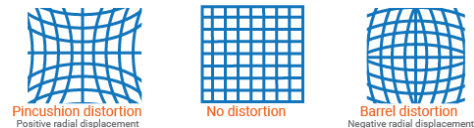
Physical reality: Distortions

Real lenses have optical imperfections, **they must be corrected before any estimation.**

1. Radial Distortion

- Caused by the curvature of the lens.
- Rays are deviated more strongly **far from the center.**
- **Consequence:** Straight lines appear **curved.**
- **Effects:** "Barrel" (GoPro, fisheye) or "Pincushion".

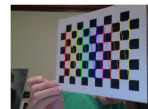
These defects are constant → corrected by calibration. The calibration parameters are found by observing a checkerboard of known size. The image is then corrected using, for example, OpenCV `cv2.undistort`.



Radial barrel distortion (left) and pincushion (right)

2. Tangential Distortion

- Caused by an assembly defect.
- The lens is not perfectly parallel to the sensor.



Calibration with a known pattern. The distances observed in the image are compared with the real distances between the corners of the checkerboard.

Stereo Vision – Disparity

How do we recover the depth Z from **two images**?

Standard configuration:

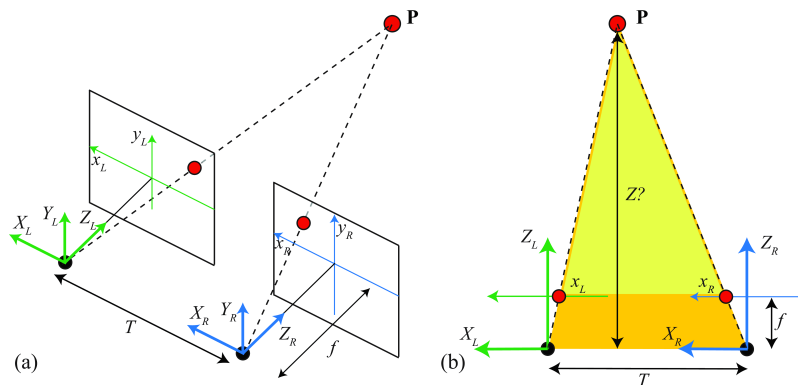
- Two identical cameras, **parallel** optical axes.
- Separated by a distance b (**baseline**).
- Focal length f (in pixels).

Disparity d : Difference in horizontal position of the same point P in the two images:

$$d = u_L - u_R$$

A **nearby** object moves a lot between the two images $\rightarrow$ large disparity.

A **distant** object moves little $\rightarrow$ small disparity.



Standard stereo geometry (Source: MIT Vision Book)

Stereo Vision – Depth

Depth relation Z : By similar triangles (Thales' theorem):

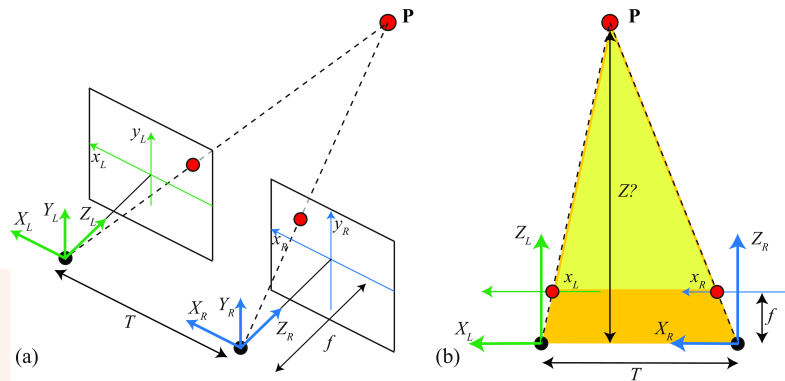
$$\frac{b}{Z} = \frac{d}{f} \implies \boxed{Z = \frac{f \cdot b}{d}}$$

Key intuition

Depth is **inversely proportional** to disparity.

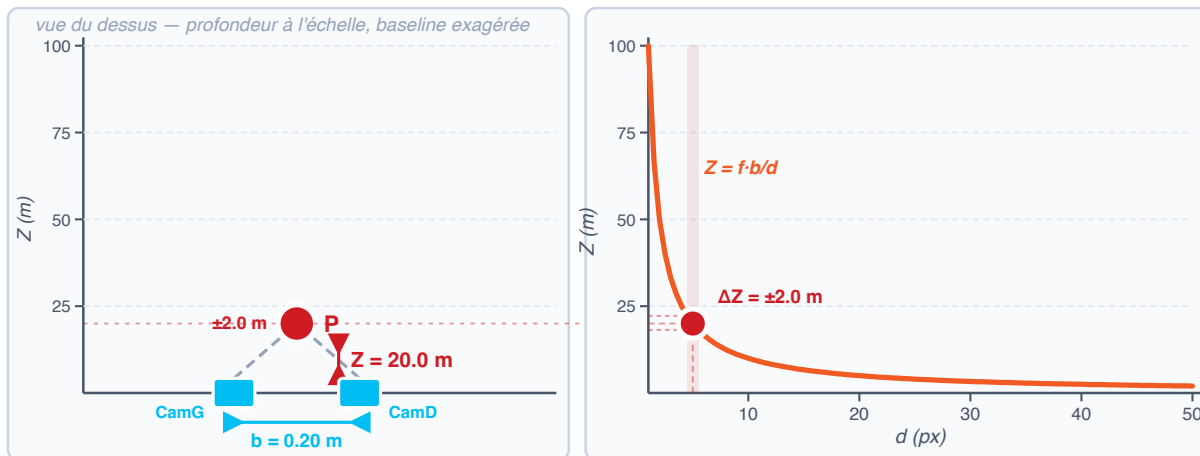
- Large $d \rightarrow$ nearby object.
- Small $d \rightarrow$ distant object (and more uncertain).

To see far with precision, you need a **large baseline b** (called T in the figure).



Standard stereo geometry (Source: MIT Vision Book)

Stereo Vision – Interactive Visualization



$f = 500$ px $b = 0.20$ m $d = 5$ px

incertitude sur $d = 0.5$ px

$z = f \cdot b / d = 20.0$ m $\Delta z = \pm 2.02$ m (10 %)

pente $|dz/dd| = z^2 / (f \cdot b) = 4.00$ m/px

RGB-D Sensors – Active Depth

Alternatives to passive stereoscopy for measuring Z directly.

1. Active IR Stereo (e.g. Intel RealSense D435)

① **IR projector** – projects a pattern of dots, invisible to the naked eye, onto the scene.

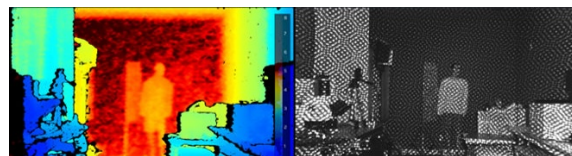
② **Two IR cameras** – see the pattern simultaneously; like two eyes, they perceive each dot slightly shifted.

③ **Depth computation** – the processor measures the *shift* of each dot between the two images $\rightarrow Z = f \cdot b/d$.

- **Limitation** : IR saturated in bright sunlight.

3. Time-of-Flight (ToF) (e.g. Kinect v2)

- Measures the phase shift of a modulated light $\rightarrow$ direct depth.



Intel RealSense D435 – active IR stereo (Source: Intel)

Advantage for SLAM

- Direct depth $\rightarrow$ no scale ambiguity in monocular.
- Eases point association, does not require features.
- Widely used indoors (domestic robot, building scan).

Visual Odometry

Feature-Based Pipeline – Overview

The classical method, still dominant in practice (ORB-SLAM, VINS-Mono):

① Detection

Find *interest points* (corners, blobs) in the image I_k .

② Description

Compute a robust descriptor vector around each point.

③ Matching

Find the correspondences between I_{k-1} and I_k .
Filter with RANSAC.

④ Estimation

Compute $T_{k-1,k}$ via a coplanarity constraint (2D-2D) or PnP (3D-2D).

Similar to ICP

We associate the pixels of the two images (steps 1 to 3).

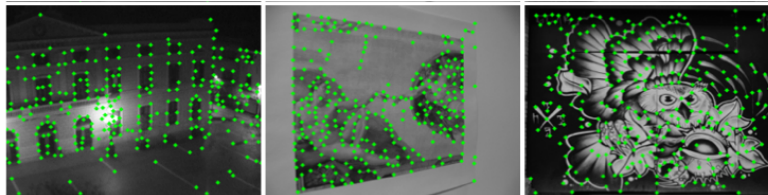
Then we compute the rotation and translation between the two images.

Feature Detectors – Detecting Interest Points

We want points that are **repeatable** and **stable** under rotation, scale change and illumination change.

FAST (*Features from Accelerated Segment Test*)

- Binary test on a circle of 16 pixels.
- Detects corners in images.
- **Very fast** for real time.



Keypoints detected on an image

Other detectors:

- **Harris**: corner response, slower.
- **SIFT / SURF**: scale invariant, expensive.
- **ORB** = FAST + orientation → **fast and invariant**.

Finding the Correspondences

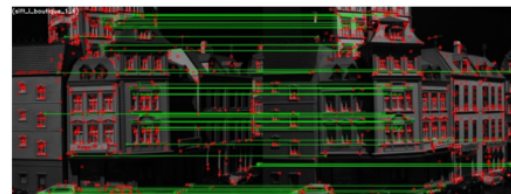
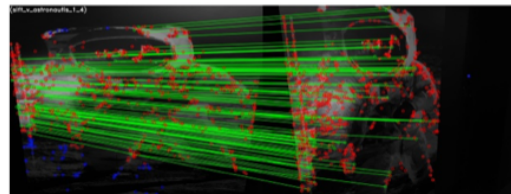
How do we know that point A in I_1 is point B in I_2 ?

BRIEF / ORB Descriptor :

- **Binary** vector (128–256 bits) built from pairwise intensity comparisons around the keypoint.
- Matching via **Hamming distance** (bitwise XOR) → very fast.
- **Major drawback**: since the descriptors are imperfect, they produce many bad correspondences.

Filtering out bad correspondences – RANSAC :

1. Pick **N points** at random.
2. Compute the model. 3. Count the **inliers**.
4. Repeat → keep the **best model**.



Raw matches (top) and after RANSAC (bottom)

Matching & RANSAC – Interactive Visualization

Image k-1

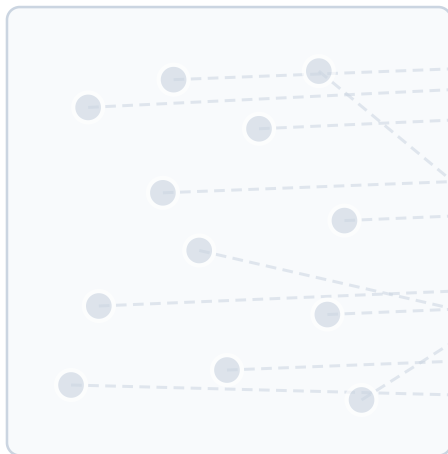


Image k



① Tirer 2 points



② Calculer modèle



③ Mesurer l'accord



④ Garder le meilleur

12 correspondances brutes : impossible de savoir lesquelles sont fausses



▶ Auto



1 / 11

Finding the transformation between two images: Coplanarity Constraint

Used for **initialization** (no 3D map yet) – input: 2D-2D correspondences.

The same 3D point, seen twice (*normalized coordinates*
 $x = K^{-1}\tilde{m}$):

$$\lambda_1 x_1 = t_2^1 + \lambda_2 R_2^1 x_2$$

The depths λ_1, λ_2 are **unknown**, but x_1, t_2^1 and $R_2^1 x_2$ lie in a **single plane**: their scalar triple product is zero.

$$x_1^T (t_2^1 \times R_2^1 x_2) = 0 \quad \Longleftrightarrow \quad \boxed{x_1^T E x_2 = 0}$$

Essential matrix $E = [t_2^1]_{\times} R_2^1$: each correspondence gives **one** equation. Several correspondences $\rightarrow E \rightarrow R$ and the **direction** of t .

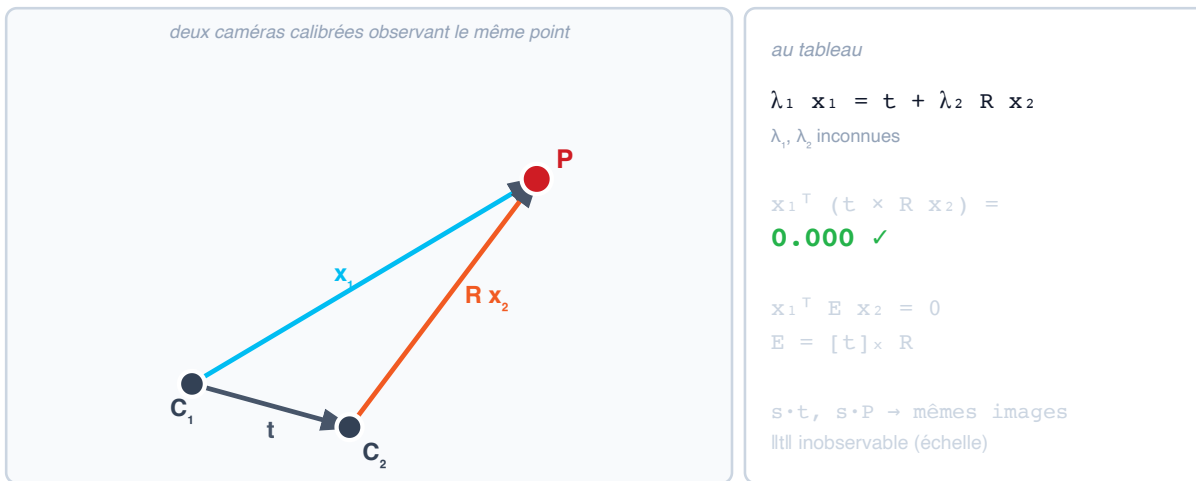
The procedure:

correspondences $(x_1, x_2) \rightarrow E \rightarrow R, t$

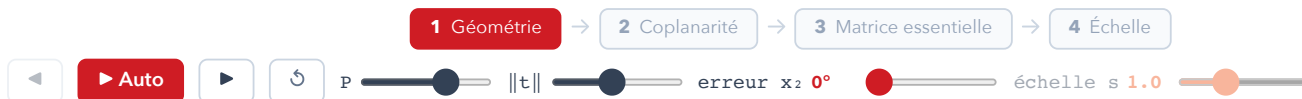
Each correspondence contributes **one** constraint on the relative pose. With several correspondences we estimate E , then **decompose** it to recover R and the **direction** of t .

```
# OpenCV – Essential Matrix + pose
E, mask = cv2.findEssentialMat(
    pts1, pts2, K, method=cv2.RANSAC)
_, R, t, _ = cv2.recoverPose(E, pts1, pts2, K)
# t is unit-norm: no absolute scale
```


Coplanarity Constraint – Interactive Visualization



Le même point 3D vu deux fois : $\lambda_1 \mathbf{x}_1 = \mathbf{t} + \lambda_2 \mathbf{R} \mathbf{x}_2$. Les deux profondeurs sont inconnues.



3D-2D: Perspective-n-Point (PnP)

Used for **tracking** (3D map available) – input: 3D points from the map + detected 2D pixels.

Where do the 3D points come from? Triangulated during initialization (2D-2D) and added to the map.

For each correspondence, in normalized coordinates $x_i = K^{-1}\tilde{m}_i$:

$$\lambda_i x_i = R_w^c p_i^w + t_w^c$$

The landmark brought into the camera frame must fall **on the ray** of its observation; its depth λ_i remains unknown.

- In practice: **solvePnP** → robust to false positives.

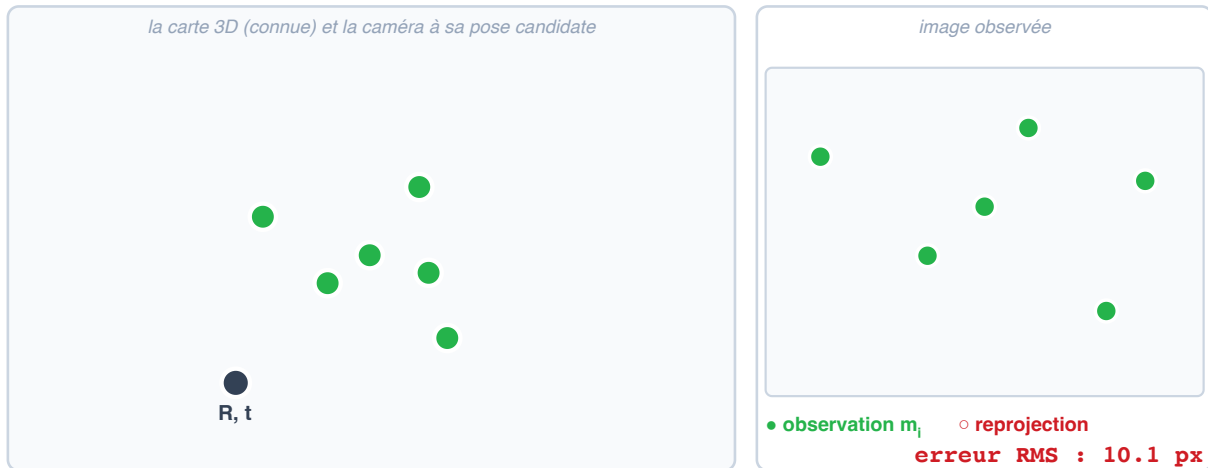
```
# OpenCV – robust PnP with RANSAC
ret, rvec, tvec, inliers = cv2.solvePnPRansac(
    pts3d,          # (N,3) float32 – map
    pts2d,          # (N,2) float32 – pixels
    K, dist_coeffs)

# Convert rotation vector → matrix
R, _ = cv2.Rodrigues(rvec)
# T = [R | t]
```

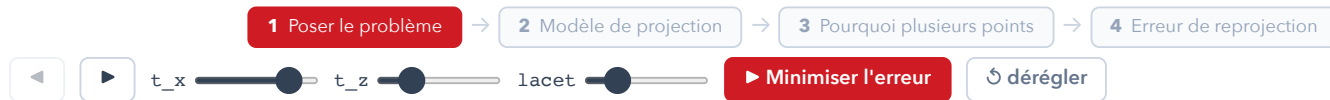
	2D-2D	PnP (3D-2D)
Input	2D ↔ 2D	3D ↔ 2D
Equation	$x_1^T E x_2 = 0$	$\lambda_i x_i = R_w^c p_i^w + t_w^c$
Output	relative pose	pose in the map
Scale	unknown	that of the map
Use	initialization	tracking

If the 3D points are in meters (e.g. via stereo) → pose with **metric scale**. If the 3D points are normalized (monocular) → pose without scale (*up-to-scale*).

PnP – Minimize the difference between the estimated camera pose and the observations.



Connus : les amers 3D de la carte et leurs pixels. Inconnue : la pose (R_w^c , t_w^c) de la caméra.



PnP – Geometric Constraint

Animation legend:

Symbol	Meaning
p_i^w	3D map landmark – known
blue ray	observation direction $x_i = K^{-1}\tilde{m}_i$, from the pixel
red segment (3D)	gap between the landmark and its ray: what the pose must cancel
● green	observation m_i in the image
○ red	reprojection $\pi(K(R_w^c p_i^w + t_w^c))$
red segment (image)	residual e_i

Objective – minimize the reprojection error:

$$\min_{R_w^c, t_w^c} \sum_{i=1}^N \|m_i - \pi(K(R_w^c p_i^w + t_w^c))\|^2$$

where $\pi(X, Y, Z) = (X/Z, Y/Z)$ is the perspective projection, without the depth λ_i .

Each correspondence (p_i^w, m_i) gives **2 equations** (u, v). The pose has **6 DOF** → at least 3 pairs are needed.

```
ret, rvec, tvec, inliers = cv2.solvePnPRansac(
    pts3d, pts2d, K, dist)
R, _ = cv2.Rodrigues(rvec)
```

Direct Methods – Alternative to Features

Instead of extracting descriptors, we directly align **pixel intensities** between images.

- No keypoint extraction → exploits all the texture, even weak texture.
- Sub-pixel precision, but sensitive to illumination changes.
- Requires **image pyramids** (coarse-to-fine) for convergence.
- Produce **semi-dense** maps (all pixels with strong gradient).

Notable direct systems

- **DSO** (Direct Sparse Odometry) – sparse direct, state of the art in speed/accuracy
- **LSD-SLAM** (Large-Scale Direct) – semi-dense, loop closure
- **SVO** (Semi-Direct VO) – hybrid: features for tracking, photometric for depth

Key takeaway

Direct methods do not replace features in every case: they excel indoors with texture, but struggle under varying lighting or outdoors.

Keyframe Selection

No need to process **every frame** – we select **keyframes**: the most informative frames.

Why keyframes?

- Consecutive frames are very similar → useless redundancy.
- BA is only run on keyframes → controlled cost.

Selection criteria:

1. **Parallax**: sufficient displacement since the last keyframe ($> \theta_{min}$).
2. **Tracking**: the number of tracked features drops below a threshold.
3. **Time**: a minimum interval between keyframes is respected.

Typical rate

At 30 fps, we select ~1 frame out of 5-10 as a keyframe.

Trade-off

Too many keyframes → expensive, slow BA. Too few → lack of constraints, drift.

VO: Pose Composition and Drift

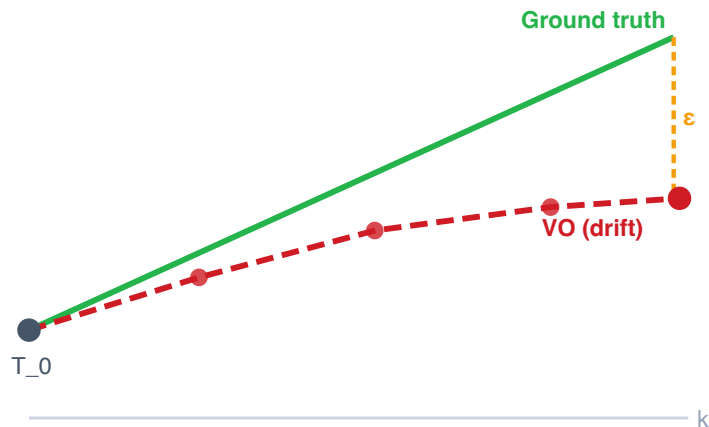
Visual odometry estimates the **relative motion** $\hat{T}_{k-1,k}$ between two consecutive frames, then **composes** the poses:

$$T_k = T_{k-1} \cdot \hat{T}_{k-1,k}$$

Each estimate $\hat{T}$ contains a small error ϵ_k . As they are composed, these errors **accumulate**, and the trajectory drifts away from reality.

VO Drift

The error grows with every new image. Without an external correction, the robot no longer knows where it is after a few hundred meters.



Drift and Local Bundle Adjustment

The problem: each estimate $\hat{T}_{k-1,k}$ has a small error $\rightarrow$ successive compositions $\rightarrow$ **drift**.

Local Bundle Adjustment: jointly re-optimizes the **last K keyframes** and their visible landmarks:

$$\min_{T_j, P_i} \sum_{j \in \mathcal{W}} \sum_i \|z_{ij} - h(T_j, P_i)\|^2$$

- The window $\mathcal{W}$ slides with the camera ($K \sim 10$).
- Cost $O(K^2)$ because every pair must be compared.

Cost vs accuracy

The larger the window, the more accurate the correction, but the higher the cost. In practice $K \leq 10$ for real time.

Approach	Accuracy	Compute cost
Plain VO (PnP only)	Low (drift $\uparrow$)	Very low
VO + local BA	Medium	Moderate
VO + global BA ($W = \infty$)	High	High

Bundle Adjustment (BA)

Bundle Adjustment (BA): Simultaneously optimizes the poses T_j **and** the 3D points P_i :

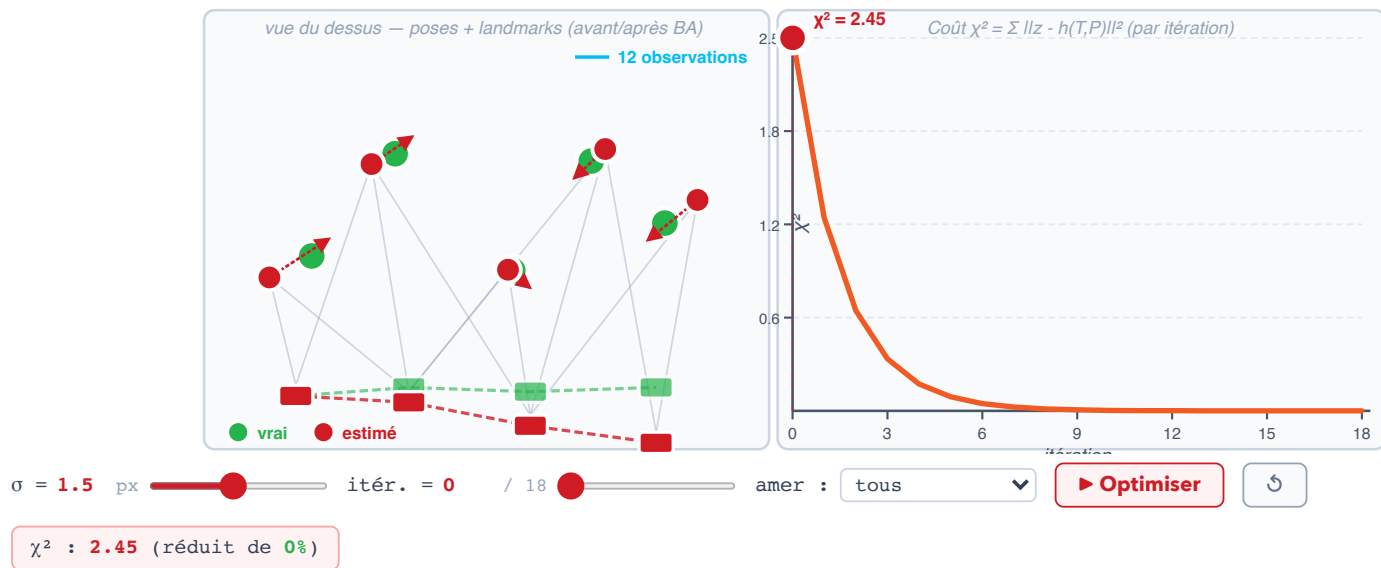
$$\min_{T,P} \sum_{i,j} \|z_{ij} - h(T_j, P_i)\|_{\Sigma_{ij}}^2$$

In practice

Local BA (window of keyframes) runs in real time in the Front-end.

Pose graph optimization over the whole trajectory (**global** BA) runs in the Back-end after each loop closure, in the next course.

Bundle Adjustment – Interactive Visualization



Limits of Visual SLAM – Why add the IMU ?

Limitations of V-SLAM alone

1. **Motion blur** : at high speed, features become unusable.
2. **Scale ambiguity** : in the monocular case.
3. **Occlusion** : facing a white wall or in the dark, the system loses tracking.
4. **Roll & Pitch** : a camera alone cannot measure the absolute orientation with respect to gravity.

Solution: add a high-frequency proprioceptive sensor.

The **IMU** (Inertial Measurement Unit) measures :

- Linear accelerations
- Angular velocities

It keeps working (> 200 Hz) **in the dark, in fog, facing a white wall.**

Visual-Inertial Odometry (VIO)

Fusing camera + IMU simultaneously resolves scale, roll/pitch and robustness to fast motion.

IMU and Inertial Navigation

Inertial Measurement Units (IMU) – Accelerometer

MEMS (Micro-Electro-Mechanical Systems) sensors. They drift quickly.

Accelerometer Model (a_m)

Measures the **specific force** in the robot frame (r):

$$a_m^r = R_w^r (a^w - g^w) + b_a + n_a$$

- a^w, g^w : True acceleration and gravity (world frame w).
- $R_w^r = (R_r^w)^T$: World $\rightarrow$ robot rotation.
- b_a, n_a : Bias (slowly varying) and white Gaussian noise.

Key point

The accelerometer measures the **specific force** $f = a - g$, **not** pure acceleration. At rest on a table, it measures $+g$ upward (normal reaction). In free fall, it measures **zero**.

Inertial Measurement Units (IMU) – Gyroscope

Gyroscope Model (ω_m)

Measures the robot's angular velocity in its own frame:

$$\omega_m^r = \omega^r + b_g + n_g$$

- ω^r : True angular velocity (expressed in the robot frame r).
- b_g , n_g : Gyroscope bias and Gaussian white noise.

Used to integrate rotation:

$$R_{k+1} = R_k \cdot \text{Exp}((\omega_k - b_g) \Delta t)$$

The gyroscope provides a very precise **relative orientation** over short durations.

We will see the Exp map (matrix operation) in the next lecture.

Camera / Gyroscope Complementarity

The camera provides an **absolute** orientation (relative to the scene) but at low frequency.

The gyroscope provides a **relative** orientation at very high frequency.

IMU: Noise and Bias – Why Does It Drift?

1. White Noise (n):

$$n \sim \mathcal{N}(0, \sigma^2)$$

During integration, the noise causes a *Random Walk* in position:

$$\sigma_p(t) \propto \sigma_n \sqrt{t}$$

2. Bias (b): A systematic error that evolves slowly (bias Random Walk):

$$\dot{b}(t) = \eta_b(t), \quad \eta_b \sim \mathcal{N}(0, \sigma_b^2)$$

Quadratic drift in position

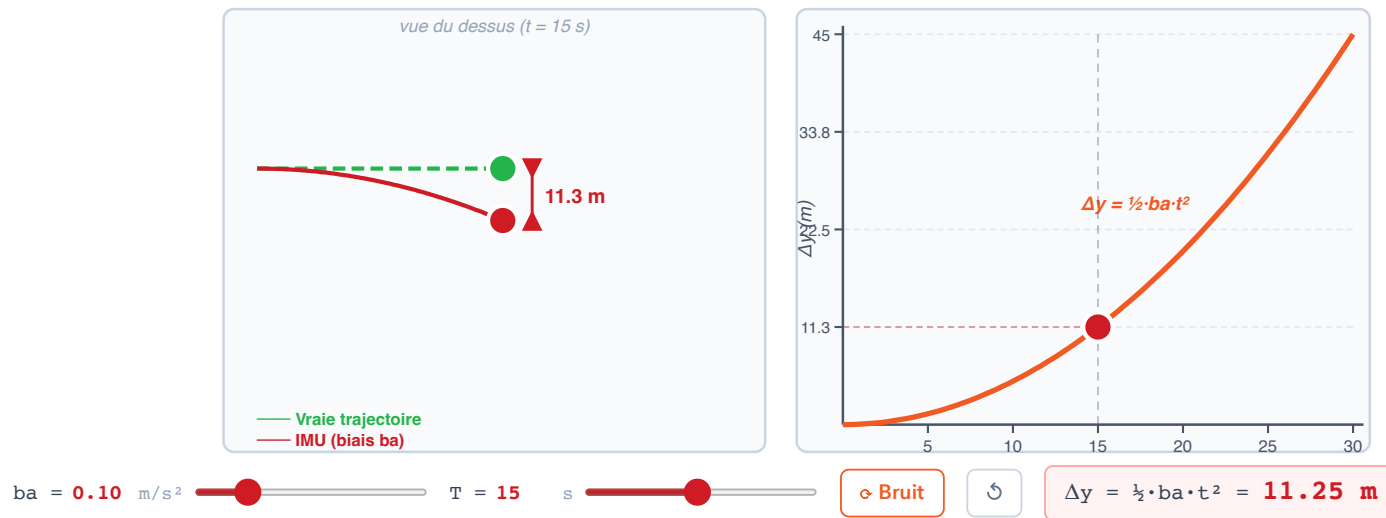
A constant bias of only **0.1 m/s²** on the accelerometer causes a position error:

$$x(t) = \frac{1}{2} b \cdot t^2$$

After **10 seconds** → an error of **5 meters**!

Estimating the biases b_a, b_g is **critical** and is performed jointly with the pose in the Back-end.

IMU Drift – Interactive Visualization



Discrete Kinematics

If we know the state $x_k = (R_k, v_k, p_k)$ and the measurements (ω_k, a_k) , we predict x_{k+1} :

Rotation:

$$R_{k+1} = R_k \cdot \text{Exp}((\omega_k - b_g) \Delta t)$$

Linear velocity:

$$v_{k+1} = v_k + (R_k (a_k - b_a) + g) \Delta t$$

Position:

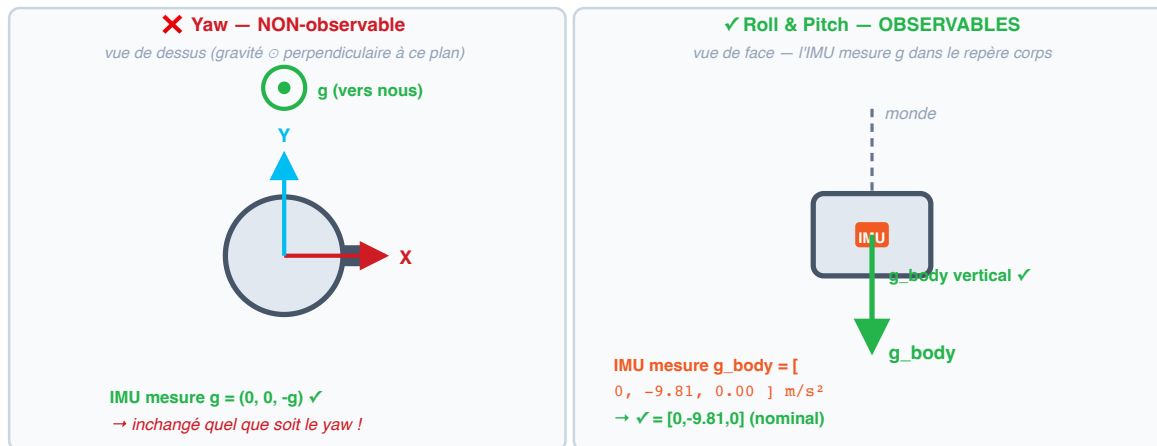
$$p_{k+1} = p_k + v_k \Delta t + \frac{1}{2} (R_k (a_k - b_a) + g) \Delta t^2$$

Key points

- The IMU runs at **very high frequency** (200-1000 Hz).
- The camera runs at low frequency (10-30 Hz).
- Between two images, we integrate **tens to hundreds** of IMU measurements.

Visual-Inertial SLAM (VIO)

IMU Observability



Yaw = 0°

non-obs.

Roll = 0°

observable

X non-observable : Tx, Ty, Tz, Yaw (4 DOF)

✓ observable : Roll, Pitch, Échelle (IMU)

VIO Advantages

Roll & Pitch Observable

Gravity g is an absolute reference measured by the accelerometer.

→ Roll and pitch are **observable**.

Metric Scale

In the monocular case, the camera estimates t in **relative units**.

The accelerometer measures in **m/s²** → it fixes the true metric scale.

→ The monocular scale ambiguity is **resolved**.

Robustness

The IMU works:

- In the **dark**.
- Facing a **white wall**.
- During **fast motions** (motion blur).

Popular VIO systems

ORB-SLAM3 (tightly coupled, multi-map) · **VINS-Mono** (preintegration) · **Kimera** (semantics + VIO)

Factor Graph – Intuition

Structure that **organizes the optimization** in the Back-end.

A **factor graph** represents the estimation problem:

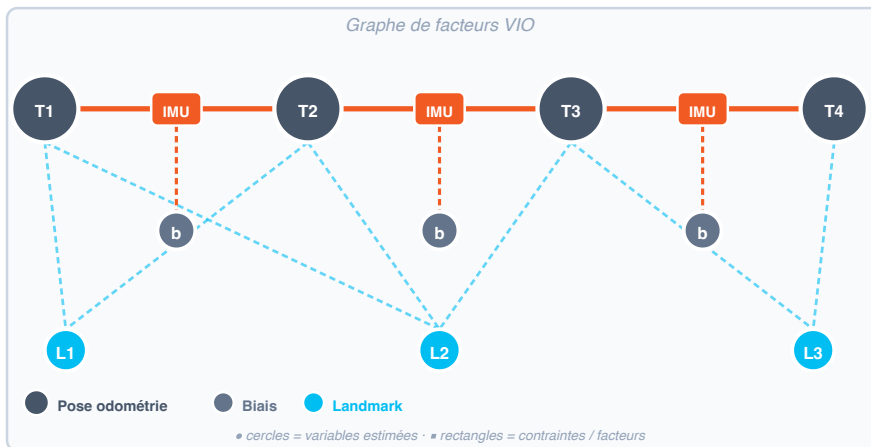
- **Nodes (variables):** what we want to estimate
poses T_i , velocities v_i , IMU biases b_i
- **Factors:** the constraints imposed by the
measurements: odometry, reprojection, IMU
- **Optimization:** find the values of the variables that
minimize the error of all constraints
simultaneously

Each factor adds a **constraint** (error to minimize). The **Back-end** (GTSAM, g2o) solves the whole set simultaneously.

Next lecture

How do we solve this system efficiently? Least squares, sparsity, Lie groups.

VIO Factor Graph – Interactive Visualization



N keyframes = 4



